

Experiment No 3

Aim:

Create a Cryptocurrency (HadCoin) using Python and perform mining in a Blockchain implemented across a peer-to-peer network of multiple nodes.

Theory:

1. Blockchain Overview

Blockchain is a distributed and decentralized ledger that stores information in a series of linked blocks. Each block contains transaction data, a timestamp, the previous block's hash, and its own unique hash. Once data is recorded, it becomes immutable, since altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of collecting pending transactions into a block, performing a computational Proof-of-Work puzzle to find a valid hash, and adding the new block to the chain. Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, three independent blockchain nodes were simulated on ports 5001, 5002, and 5003. Each node runs on a separate port, maintains its own copy of the blockchain, and can connect with peers to share and validate blocks.

4. Consensus Mechanism

The Longest Chain Rule was used: if multiple versions of the chain exist across nodes, the longest valid chain is chosen, ensuring all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction contains a sender, receiver, and amount. When a block is mined, all pending transactions are added to the block, and a reward transaction is automatically created to pay the miner. After mining, the transaction pool (mempool) is cleared.

6. Chain Replacement

When `/replace_chain` is called, a node requests chains from its peers; if a longer valid chain is found, it replaces its own — keeping the blockchain consistent across the network.

Output -

Body Cookies Headers 5 Test Results | ↺

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   "amount": 50,
3   "block_index": 2,
4   "message": "Siddharth, transaction added successfully.",
5   "receiver": "Rahul",
6   "sender": "Siddharth"
7 }
8
```

```
{
  "index": 2,
  "message": "Congratulations Siddharth! HadCoin block mined successfully.",
  "miner": "e683c86176ea47638099f76df32f7dbb",
  "previous_hash": "16f29e4d6afcc469d6a91d6a8fdb43f0f2ad5a77c9052873da85098d04c7c91e",
  "proof": 533,
  "timestamp": 1788939760.0597084,
  "transactions": [
    {
      "amount": 50,
      "receiver": "Rahul",
      "sender": "Siddharth"
    },
    {
      "amount": 10,
      "receiver": "e683c86176ea47638099f76df32f7dbb",
      "sender": "0"
    }
  ]
}
```

```

{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": 1788939613.3396556,
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "16f29e4d6afcc469d6a91d6a8fdb43f0f2ad5a77c9052873da85098d04c7c91e",
      "proof": 533,
      "timestamp": 1788939760.0597084,
      "transactions": [
        {
          "amount": 50,
          "receiver": "Rahul",
          "sender": "Siddharth"
        },
        {
          "amount": 10,
          "receiver": "e683c86176ea47638099f76df32f7dbb",
          "sender": "0"
        }
      ]
    }
  ],
  "length": 2,
  "message": "Siddharth, here is the HadCoin blockchain."
}

```

```

1  {
2    "message": "Siddharth, the HadCoin blockchain is valid.",
3    "valid": true
4  }
5
6  {
7    "message": "Siddharth, nodes connected successfully.",
8    "total_nodes": [
9      "127.0.0.1:5002",
10     "127.0.0.1:5003"
11   ]
12 }

```

```

{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": 1788939613.3396556,
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "16f29e4d6afcc469d6a91d6a8fdb43f0f2ad5a77c9052873da85098d04c7c91e",
      "proof": 533,
      "timestamp": 1788939760.0597084,
      "transactions": [
        {
          "amount": 50,
          "receiver": "Rahul",
          "sender": "Siddharth"
        },
        {
          "amount": 10,
          "receiver": "e683c86176ea47638099f76df32f7dbb",
          "sender": "0"
        }
      ]
    }
  ],
  "message": "Siddharth, the current chain is already the longest valid chain."
}

```

```

{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": 1788939613.3396556,
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "16f29e4d6afcc469d6a91d6a8fdb43f0f2ad5a77c9052873da85098d04c7c91e",
      "proof": 533,
      "timestamp": 1788939760.0597084,
      "transactions": [
        {
          "amount": 50,
          "receiver": "Rahul",
          "sender": "Siddharth"
        },
        {
          "amount": 10,
          "receiver": "e683c86176ea47638099f76df32f7dbb",
          "sender": "0"
        }
      ]
    }
  ],
  "length": 2,
  "message": "Siddharth, here is the HadCoin blockchain."
}

```

Conclusion -

We implemented a cryptocurrency (HadCoin) in Python using Flask, simulating a three-node peer-to-peer blockchain network with Proof-of-Work mining and automatic miner rewards. Node 5001 mined a block, creating an intentional chain-length mismatch across nodes, which was resolved when nodes 5002 and 5003 synchronized via the Longest Chain Rule using `/replace_chain`. This demonstrated the core blockchain concepts of decentralized consensus, transaction validation, and network-wide synchronization.